

November 29, 2025

1 Data Cleaning and Filtering

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[2]: # Load the complete NBA player statistics dataset
df = pd.read_csv('../data/02-player-stats-extraction/nba_complete_player_stats.
↪csv')
```

```
[3]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 35646 entries, 0 to 35645
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   player_name           35646 non-null  object
1   year_id               35646 non-null  object
2   age                  32584 non-null  float64
3   team_name_abbrev     35646 non-null  object
4   games                32604 non-null  float64
5   mp                  31525 non-null  float64
6   per                  31508 non-null  float64
7   bpm                  26938 non-null  float64
8   vorp                 26943 non-null  float64
9   ws                   32593 non-null  float64
10  ws_per_48             31508 non-null  float64
dtypes: float64(8), object(3)
memory usage: 3.0+ MB
```

```
[4]: df.head()
```

```
[4]:   player_name  year_id  age  team_name_abbrev  games   mp   per  bpm  \
0  Alaa Abdelnaby  1990-91  22.0             POR    43.0  290.0  13.1 -4.6
1  Alaa Abdelnaby  1991-92  23.0             POR    71.0  934.0  13.5 -2.6
2  Alaa Abdelnaby  1992-93  24.0             2TM    75.0  1311.0  13.2 -3.9
```

3	Alaa Abdelnaby	1992-93	24.0	MIL	12.0	159.0	11.9	-3.5
4	Alaa Abdelnaby	1992-93	24.0	BOS	63.0	1152.0	13.4	-4.0

	vorp	ws	ws_per_48
0	-0.2	0.5	0.079
1	-0.1	2.1	0.110
2	-0.6	2.0	0.074
3	-0.1	0.1	0.035
4	-0.6	1.9	0.079

Let's look at the seasons past the ABA-NBA merger in 1976.

```
[5]: print(f"Min year: {df['year_id'].min()}")
      print(f"Max year: {df['year_id'].max()}")
```

Min year: 1946-47

Max year: 2024-25

```
[6]: # Create a new column with the start year of the season
      df['start_year'] = df['year_id'].str[:4].astype(int)

      df = df[df['start_year'] >= 1976]

      # Drop the start_year column
      df = df.drop('start_year', axis=1)

      df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 28126 entries, 0 to 35643
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   player_name      28126 non-null  object
1   year_id          28126 non-null  object
2   age              25665 non-null  float64
3   team_name_abbr   28126 non-null  object
4   games            25665 non-null  float64
5   mp               25665 non-null  float64
6   per              25660 non-null  float64
7   bpm              25660 non-null  float64
8   vorp             25665 non-null  float64
9   ws               25665 non-null  float64
10  ws_per_48        25660 non-null  float64
dtypes: float64(8), object(3)
memory usage: 2.6+ MB
```

We only care about BPM the most

```
[12]: df = df.dropna(subset=['bpm'])
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 25660 entries, 0 to 35643
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   player_name           25660 non-null  object
1   year_id               25660 non-null  object
2   age                  25660 non-null  float64
3   team_name_abbr       25660 non-null  object
4   games                25660 non-null  float64
5   mp                   25660 non-null  float64
6   per                  25660 non-null  float64
7   bpm                  25660 non-null  float64
8   vorp                 25660 non-null  float64
9   ws                   25660 non-null  float64
10  ws_per_48             25660 non-null  float64
dtypes: float64(8), object(3)
memory usage: 2.3+ MB
```

```
[13]: df.head()
```

```
[13]:      player_name  year_id  age team_name_abbr  games    mp    per  bpm  \
0  Alaa Abdelnaby  1990-91  22.0             POR   43.0  290.0  13.1 -4.6
1  Alaa Abdelnaby  1991-92  23.0             POR   71.0  934.0  13.5 -2.6
2  Alaa Abdelnaby  1992-93  24.0             2TM   75.0 1311.0  13.2 -3.9
3  Alaa Abdelnaby  1992-93  24.0             MIL   12.0  159.0  11.9 -3.5
4  Alaa Abdelnaby  1992-93  24.0             BOS   63.0 1152.0  13.4 -4.0

      vorp    ws  ws_per_48
0   -0.2   0.5    0.079
1   -0.1   2.1    0.110
2   -0.6   2.0    0.074
3   -0.1   0.1    0.035
4   -0.6   1.9    0.079
```

```
[14]: # Analyze missing values and data quality
print("="*60)
print("MISSING VALUES ANALYSIS")
print("="*60)

# Check for missing values in each column
missing_values = df.isnull().sum()
missing_percent = (missing_values / len(df)) * 100

missing_df = pd.DataFrame({
```

```

    'Column': missing_values.index,
    'Missing_Count': missing_values.values,
    'Missing_Percent': missing_percent.values
}).sort_values('Missing_Count', ascending=False)

print(missing_df)

print("\n" + "="*60)
print("DATA QUALITY ISSUES")
print("="*60)

# Check for problematic values
print("1. Records with missing age:")
missing_age = df[df['age'].isnull()]
print(f"    Count: {len(missing_age)}")
if len(missing_age) > 0:
    print(f"    Sample players: {missing_age['player_name'].head().tolist()}")

print("\n2. Records with 'Did not play' entries:")
dnp_records = df[df['team_name_abbr'].str.contains('Did not play', na=False)]
print(f"    Count: {len(dnp_records)}")
if len(dnp_records) > 0:
    print(f"    Sample: {dnp_records['player_name'].head().tolist()}")

print("\n3. Records with zero or negative games:")
zero_games = df[(df['games'] <= 0) | (df['games'].isnull())]
print(f"    Count: {len(zero_games)}")

print("\n4. Records with zero or negative minutes:")
zero_minutes = df[(df['mp'] <= 0) | (df['mp'].isnull())]
print(f"    Count: {len(zero_minutes)}")

print("\n5. Advanced stats availability by era:")
# Check when advanced stats became available
stats_by_year = df.groupby('year_id').agg({
    'per': lambda x: x.notna().sum(),
    'bpm': lambda x: x.notna().sum(),
    'vorp': lambda x: x.notna().sum(),
    'ws': lambda x: x.notna().sum(),
    'player_name': 'count'
}).rename(columns={'player_name': 'total_records'})

stats_by_year['per_pct'] = (stats_by_year['per'] /
    ↪stats_by_year['total_records']) * 100
stats_by_year['bpm_pct'] = (stats_by_year['bpm'] /
    ↪stats_by_year['total_records']) * 100

```

```

print("    Year ranges with advanced stats:")
print(f"    PER available from: {stats_by_year[stats_by_year['per_pct'] > 90].
    ↪index.min()}")
print(f"    BPM available from: {stats_by_year[stats_by_year['bpm_pct'] > 90].
    ↪index.min()}")

# Show first few years of data
print("\n    First 10 years stats availability:")
print(stats_by_year.head(10)[['total_records', 'per_pct', 'bpm_pct']].round(1))

```

MISSING VALUES ANALYSIS

	Column	Missing_Count	Missing_Percent
0	player_name	0	0.0
1	year_id	0	0.0
2	age	0	0.0
3	team_name_abbr	0	0.0
4	games	0	0.0
5	mp	0	0.0
6	per	0	0.0
7	bpm	0	0.0
8	vorp	0	0.0
9	ws	0	0.0
10	ws_per_48	0	0.0

DATA QUALITY ISSUES

1. Records with missing age:
Count: 0
2. Records with 'Did not play' entries:
Count: 0
3. Records with zero or negative games:
Count: 0
4. Records with zero or negative minutes:
Count: 0
5. Advanced stats availability by era:
Year ranges with advanced stats:
PER available from: 1976-77
BPM available from: 1976-77

First 10 years stats availability:

year_id	total_records	per_pct	bpm_pct
1976-77	355	100.0	100.0
1977-78	372	100.0	100.0
1978-79	342	100.0	100.0
1979-80	356	100.0	100.0
1980-81	362	100.0	100.0
1981-82	372	100.0	100.0
1982-83	388	100.0	100.0
1983-84	343	100.0	100.0
1984-85	362	100.0	100.0
1985-86	379	100.0	100.0

```
[16]: # Data Cleaning and Standardization
print("="*60)
print("CLEANING AND STANDARDIZING DATASET")
print("="*60)

# Create a copy for cleaning
df_clean = df.copy()

print(f"Starting with {len(df_clean):,} records")

# 1. Remove "Did not play" records
print("\n1. Removing 'Did not play' records...")
before_dnp = len(df_clean)
df_clean = df_clean[~df_clean['team_name_abbrev'].str.contains('Did not play',
    na=False)]
after_dnp = len(df_clean)
print(f"    Removed {before_dnp - after_dnp:,} records")

# 2. Remove records with missing or zero games
print("\n2. Removing records with missing/zero games...")
before_games = len(df_clean)
df_clean = df_clean[(df_clean['games'] > 0) & (df_clean['games'].notna())]
after_games = len(df_clean)
print(f"    Removed {before_games - after_games:,} records")

# 3. Remove records with missing or zero minutes
print("\n3. Removing records with missing/zero minutes...")
before_minutes = len(df_clean)
df_clean = df_clean[(df_clean['mp'] > 0) & (df_clean['mp'].notna())]
after_minutes = len(df_clean)
print(f"    Removed {before_minutes - after_minutes:,} records")

# 4. Handle missing age values
print("\n4. Handling missing age values...")
```

```

missing_age_count = df_clean['age'].isnull().sum()
print(f"    Records with missing age: {missing_age_count}")

if missing_age_count > 0:
    print("    Sample players with missing age:")
    print(df_clean[df_clean['age'].isnull()][['player_name']].head().tolist())

    # For now, remove records with missing age
    df_clean = df_clean[df_clean['age'].notna()]
    print(f"    Removed {missing_age_count} records with missing age")

# 5. Convert data types
print("\n5. Converting data types...")

# Function to extract the 4-digit season end year (e.g., '1990-91' -> 1991)
def get_season_end_year(year_id):
    # Extracts the century (e.g., '19' from '1990')
    century = year_id[:2]
    # Extracts the end year's two digits (e.g., '91' from '1990-91')
    end_year_two_digits = year_id.split('-')[1]
    return int(century + end_year_two_digits)

# Apply the function to correctly convert the season ID to the end year
df_clean['year_id'] = df_clean['year_id'].apply(get_season_end_year)

# Convert remaining types
df_clean['games'] = df_clean['games'].astype(int)
df_clean['age'] = df_clean['age'].astype(int)

# 6. Handle duplicate player-season records (e.g., trades)
print("\n6. Handling duplicate player-season records...")
duplicates = df_clean[df_clean.duplicated(subset=['player_name', 'year_id'],
    ↪keep=False)]
print(f"    Found {len(duplicates)} duplicate player-season records")

# For players with multiple teams in same season, keep the combined "2TM"
    ↪record if available
# Otherwise keep the record with most games played
def handle_duplicates(group):
    if len(group) == 1:
        return group

    # Prefer "2TM" (two team) or similar multi-team records
    multi_team = group[group['team_name_abbr'].str.contains('TM', na=False)]
    if len(multi_team) > 0:
        return multi_team.iloc[[0]] # Keep first multi-team record

```

```

    # Otherwise keep record with most games
    return group.loc[group['games'].idxmax()].to_frame().T

df_clean = df_clean.groupby(['player_name', 'year_id']).
    ↪ apply(handle_duplicates).reset_index(drop=True)

print(f"\nCleaning complete!")
print(f"Final dataset: {len(df_clean):,} records")
print(f"Unique players: {df_clean['player_name'].nunique():,}")
print(f>Date range: {df_clean['year_id'].min()} to {df_clean['year_id'].max()}")

# Summary of cleaned dataset
print("\n" + "="*60)
print("CLEANED DATASET SUMMARY")
print("="*60)
print(f"Records: {len(df_clean):,}")
print(f"Players: {df_clean['player_name'].nunique():,}")
print(f"Years: {df_clean['year_id'].nunique():,}")
print(f"Teams: {df_clean['team_name_abbr'].nunique():,}")

# Check remaining missing values
print(f"\nRemaining missing values:")
remaining_missing = df_clean.isnull().sum()
print(remaining_missing[remaining_missing > 0])

```

=====

CLEANING AND STANDARDIZING DATASET

=====

Starting with 25,660 records

1. Removing 'Did not play' records...
Removed 0 records
2. Removing records with missing/zero games...
Removed 0 records
3. Removing records with missing/zero minutes...
Removed 0 records
4. Handling missing age values...
Records with missing age: 0
5. Converting data types...
6. Handling duplicate player-season records...
Found 7248 duplicate player-season records

Cleaning complete!

Final dataset: 20,780 records
Unique players: 3,846
Date range: 1900 to 2025

=====

CLEANED DATASET SUMMARY

=====

Records: 20,780
Players: 3,846
Years: 49
Teams: 47

Remaining missing values:
Series([], dtype: int64)

/var/folders/w6/rkc9m9295k53xz40pqjj64nm0000gn/T/ipykernel_82456/3268917802.py:8
2: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns.
This behavior is deprecated, and in a future version of pandas the grouping
columns will be excluded from the operation. Either pass `include\_groups=False`
to exclude the groupings or explicitly select the grouping columns after groupby
to silence this warning.

```
df_clean = df_clean.groupby(['player_name',  
'year_id']).apply(handle_duplicates).reset_index(drop=True)
```

```
[18]: # Advanced Stats Availability Analysis
print("="*60)
print("ADVANCED STATS AVAILABILITY BY ERA")
print("="*60)

# Calculate stats availability by year
stats_availability = df_clean.groupby('year_id').agg({
    'per': lambda x: x.notna().sum(),
    'bpm': lambda x: x.notna().sum(),
    'vorp': lambda x: x.notna().sum(),
    'ws': lambda x: x.notna().sum(),
    'ws_per_48': lambda x: x.notna().sum(),
    'player_name': 'count'
}).rename(columns={'player_name': 'total_records'})

# Calculate percentages
for stat in ['per', 'bpm', 'vorp', 'ws', 'ws_per_48']:
    stats_availability[f'{stat}_pct'] = (stats_availability[stat] /
    stats_availability['total_records']) * 100

print("Advanced stats introduction timeline:")
print("=====")

# Find when each stat became widely available (>90% coverage)
```

```

stat_intro_years = {}
for stat in ['per', 'bpm', 'vorp', 'ws', 'ws_per_48']:
    available_years = stats_availability[stats_availability[f'{stat}_pct'] > 90]
    if len(available_years) > 0:
        intro_year = available_years.index.min()
        stat_intro_years[stat] = intro_year
        print(f"{stat.upper()}: {intro_year} onwards ({stats_availability.
↳ loc[intro_year, f'{stat}_pct']:.1f}% coverage)")
    else:
        print(f"{stat.upper()}: Not widely available")

# Show timeline visualization
print(f"\nStats availability timeline (first 20 years):")
print(stats_availability.head(20)[['total_records', 'per_pct', 'bpm_pct', '
↳ ws_pct']].round(1))

# Create era definitions based on stat availability
print(f"\n" + "="*60)
print("ERA DEFINITIONS FOR ANALYSIS")
print("="*60)

# Define eras based on advanced stats availability
eras = {
    'Pre-Advanced (1946-1973)': {'start': 1946, 'end': 1973, 'stats': ['basic_
↳ stats only']},
    'Early Advanced (1974-1977)': {'start': 1974, 'end': 1977, 'stats': ['basic_
↳ stats', 'some advanced']},
    'PER Era (1978-1992)': {'start': 1978, 'end': 1992, 'stats': ['PER', 'Win_
↳ Shares']},
    'Full Advanced (1993-2024)': {'start': 1993, 'end': 2024, 'stats': ['PER', '
↳ BPM', 'VORP', 'Win Shares']}
}

for era_name, era_info in eras.items():
    era_data = df_clean[
        (df_clean['year_id'] >= era_info['start']) &
        (df_clean['year_id'] <= era_info['end'])
    ]

    print(f"\n{era_name}:")
    print(f"  Years: {era_info['start']}--{era_info['end']}")
    print(f"  Records: {len(era_data):,}")
    print(f"  Players: {era_data['player_name'].nunique():,}")
    print(f"  Available stats: {' '.join(era_info['stats'])}")

# Check actual stat availability in this era

```

```

if len(era_data) > 0:
    per_available = era_data['per'].notna().sum()
    bpm_available = era_data['bpm'].notna().sum()
    ws_available = era_data['ws'].notna().sum()

    print(f" PER coverage: {per_available}/{len(era_data)}\n
↳({per_available/len(era_data)*100:.1f}%)")
    print(f" BPM coverage: {bpm_available}/{len(era_data)}\n
↳({bpm_available/len(era_data)*100:.1f}%)")
    print(f" WS coverage: {ws_available}/{len(era_data)}\n
↳({ws_available/len(era_data)*100:.1f}%)")

# Add era column to dataset
def assign_era(year):
    if year <= 1973:
        return 'Pre-Advanced'
    elif year <= 1977:
        return 'Early Advanced'
    elif year <= 1992:
        return 'PER Era'
    else:
        return 'Full Advanced'

df_clean['era'] = df_clean['year_id'].apply(assign_era)

print(f"\nEra distribution in cleaned dataset:")
print(df_clean['era'].value_counts())

```

```

=====
ADVANCED STATS AVAILABILITY BY ERA
=====

```

Advanced stats introduction timeline:

```
=====
```

PER: 1900 onwards (100.0% coverage)
BPM: 1900 onwards (100.0% coverage)
VORP: 1900 onwards (100.0% coverage)
WS: 1900 onwards (100.0% coverage)
WS_PER_48: 1900 onwards (100.0% coverage)

Stats availability timeline (first 20 years):

	total_records	per_pct	bpm_pct	ws_pct
year_id				
1900	438	100.0	100.0	100.0
1977	293	100.0	100.0	100.0
1978	283	100.0	100.0	100.0
1979	277	100.0	100.0	100.0
1980	285	100.0	100.0	100.0

1981	302	100.0	100.0	100.0
1982	313	100.0	100.0	100.0
1983	313	100.0	100.0	100.0
1984	309	100.0	100.0	100.0
1985	317	100.0	100.0	100.0
1986	322	100.0	100.0	100.0
1987	334	100.0	100.0	100.0
1988	331	100.0	100.0	100.0
1989	352	100.0	100.0	100.0
1990	380	100.0	100.0	100.0
1991	386	100.0	100.0	100.0
1992	386	100.0	100.0	100.0
1993	390	100.0	100.0	100.0
1994	403	100.0	100.0	100.0
1995	406	100.0	100.0	100.0

=====

ERA DEFINITIONS FOR ANALYSIS

=====

Pre-Advanced (1946-1973):

Years: 1946-1973
Records: 0
Players: 0
Available stats: basic stats only

Early Advanced (1974-1977):

Years: 1974-1977
Records: 293
Players: 293
Available stats: basic stats, some advanced
PER coverage: 293/293 (100.0%)
BPM coverage: 293/293 (100.0%)
WS coverage: 293/293 (100.0%)

PER Era (1978-1992):

Years: 1978-1992
Records: 4,890
Players: 1,190
Available stats: PER, Win Shares
PER coverage: 4,890/4,890 (100.0%)
BPM coverage: 4,890/4,890 (100.0%)
WS coverage: 4,890/4,890 (100.0%)

Full Advanced (1993-2024):

Years: 1993-2024
Records: 14,590
Players: 2,860

Available stats: PER, BPM, VORP, Win Shares
PER coverage: 14,590/14,590 (100.0%)
BPM coverage: 14,590/14,590 (100.0%)
WS coverage: 14,590/14,590 (100.0%)

Era distribution in cleaned dataset:

```
era
Full Advanced      15159
PER Era            4890
Pre-Advanced       438
Early Advanced     293
Name: count, dtype: int64
```

```
[19]: # Apply Filtering Criteria for Peak Analysis
print("="*60)
print("APPLYING FILTERING CRITERIA")
print("="*60)

# Define filtering criteria
MIN_GAMES_PER_SEASON = 20 # Minimum games to be considered a meaningful season
MIN_MINUTES_PER_GAME = 10 # Minimum minutes per game
MIN_SEASONS_CAREER = 3 # Minimum seasons in career for peak analysis
MIN_TOTAL_GAMES = 82 # Minimum total games in career

print(f"Filtering criteria:")
print(f"- Minimum games per season: {MIN_GAMES_PER_SEASON}")
print(f"- Minimum minutes per game: {MIN_MINUTES_PER_GAME}")
print(f"- Minimum seasons in career: {MIN_SEASONS_CAREER}")
print(f"- Minimum total games in career: {MIN_TOTAL_GAMES}")

# Start with cleaned dataset
df_filtered = df_clean.copy()

print(f"\nStarting with {len(df_filtered):,} records")

# 1. Filter by minimum games per season
print(f"\n1. Filtering by minimum games per season ({MIN_GAMES_PER_SEASON})...")
before_games = len(df_filtered)
df_filtered = df_filtered[df_filtered['games'] >= MIN_GAMES_PER_SEASON]
after_games = len(df_filtered)
print(f"    Removed {before_games - after_games:,} records")

# 2. Filter by minimum minutes per game
print(f"\n2. Filtering by minimum minutes per game ({MIN_MINUTES_PER_GAME})...")
before_mpg = len(df_filtered)
df_filtered['mpg'] = df_filtered['mp'] / df_filtered['games']
df_filtered = df_filtered[df_filtered['mpg'] >= MIN_MINUTES_PER_GAME]
```

```

after_mpg = len(df_filtered)
print(f"    Removed {before_mpg - after_mpg:,} records")

# 3. Calculate career statistics for each player
print(f"\n3. Calculating career statistics...")
career_stats = df_filtered.groupby('player_name').agg({
    'year_id': 'count', # Number of seasons
    'games': 'sum',     # Total games
    'mp': 'sum',        # Total minutes
    'age': ['min', 'max'] # Age range
}).round(1)

career_stats.columns = ['seasons', 'total_games', 'total_minutes', 'first_age',
    ↪ 'last_age']
career_stats['career_length'] = career_stats['last_age'] -
    ↪ career_stats['first_age'] + 1

print(f"    Career statistics calculated for {len(career_stats):,} players")

# 4. Filter by minimum seasons
print(f"\n4. Filtering by minimum seasons ({MIN_SEASONS_CAREER})...")
qualified_players = career_stats[career_stats['seasons'] >= MIN_SEASONS_CAREER].
    ↪ index
before_seasons = len(df_filtered)
df_filtered = df_filtered[df_filtered['player_name'].isin(qualified_players)]
after_seasons = len(df_filtered)
print(f"    Removed {before_seasons - after_seasons:,} records")

# 5. Filter by minimum total games
print(f"\n5. Filtering by minimum total games ({MIN_TOTAL_GAMES})...")
qualified_players = career_stats[career_stats['total_games'] >=
    ↪ MIN_TOTAL_GAMES].index
before_total_games = len(df_filtered)
df_filtered = df_filtered[df_filtered['player_name'].isin(qualified_players)]
after_total_games = len(df_filtered)
print(f"    Removed {before_total_games - after_total_games:,} records")

# Update career stats with filtered data
career_stats_filtered = df_filtered.groupby('player_name').agg({
    'year_id': 'count',
    'games': 'sum',
    'mp': 'sum',
    'age': ['min', 'max']
}).round(1)

career_stats_filtered.columns = ['seasons', 'total_games', 'total_minutes',
    ↪ 'first_age', 'last_age']

```

```

career_stats_filtered['career_length'] = career_stats_filtered['last_age'] -
    career_stats_filtered['first_age'] + 1

print(f"\nFiltering complete!")
print(f"Final filtered dataset: {len(df_filtered):,} records")
print(f"Qualified players: {df_filtered['player_name'].nunique():,}")

# Summary statistics
print(f"\n" + "="*60)
print("FILTERED DATASET SUMMARY")
print("="*60)
print(f"Records: {len(df_filtered):,}")
print(f"Players: {df_filtered['player_name'].nunique():,}")
print(f"Years: {df_filtered['year_id'].min()} to {df_filtered['year_id'].
    max()}")

print(f"\nCareer length distribution:")
print(career_stats_filtered['seasons'].describe())

print(f"\nGames per season distribution:")
print(df_filtered['games'].describe())

print(f"\nMinutes per game distribution:")
print(df_filtered['mpg'].describe())

print(f"\nEra distribution (filtered):")
print(df_filtered['era'].value_counts())

# Check advanced stats availability in filtered dataset
print(f"\nAdvanced stats availability (filtered):")
for stat in ['per', 'bpm', 'vorp', 'ws', 'ws_per_48']:
    available = df_filtered[stat].notna().sum()
    total = len(df_filtered)
    print(f"{stat.upper():,}: {available:,}/{total:,} ({available/total*100:.
        1f}%)")

```

=====

APPLYING FILTERING CRITERIA

=====

Filtering criteria:

- Minimum games per season: 20
- Minimum minutes per game: 10
- Minimum seasons in career: 3
- Minimum total games in career: 82

Starting with 20,780 records

1. Filtering by minimum games per season (20)..
Removed 3,189 records
2. Filtering by minimum minutes per game (10)..
Removed 1,787 records
3. Calculating career statistics..
Career statistics calculated for 2,667 players
4. Filtering by minimum seasons (3)..
Removed 1,166 records
5. Filtering by minimum total games (82)..
Removed 6 records

Filtering complete!

Final filtered dataset: 14,632 records

Qualified players: 1,821

```
=====
FILTERED DATASET SUMMARY
=====
```

Records: 14,632

Players: 1,821

Years: 1900 to 2025

Career length distribution:

count	1821.000000
mean	8.035146
std	4.018211
min	3.000000
25%	5.000000
50%	7.000000
75%	11.000000
max	22.000000

Name: seasons, dtype: float64

Games per season distribution:

count	14632
unique	67
top	82
freq	1627

Name: games, dtype: int64

Minutes per game distribution:

count	14632.0
unique	11792.0
top	34.0

```
freq          16.0
Name: mpg, dtype: float64
```

Era distribution (filtered):

```
era
Full Advanced      10596
PER Era            3550
Pre-Advanced       321
Early Advanced     165
Name: count, dtype: int64
```

Advanced stats availability (filtered):

```
PER: 14,632/14,632 (100.0%)
BPM: 14,632/14,632 (100.0%)
VORP: 14,632/14,632 (100.0%)
WS: 14,632/14,632 (100.0%)
WS_PER_48: 14,632/14,632 (100.0%)
```

```
[20]: # Create Analysis-Ready Datasets
print("="*60)
print("CREATING ANALYSIS-READY DATASETS")
print("="*60)

# Create different datasets for different types of analysis

# 1. MODERN ERA DATASET (1993-2024) - Full advanced stats available
print("1. Creating Modern Era Dataset (1993-2024)...")
df_modern = df_filtered[df_filtered['era'] == 'Full Advanced'].copy()

# Remove records with missing key advanced stats
df_modern = df_modern.dropna(subset=['per', 'bpm', 'ws', 'ws_per_48'])

print(f"   Records: {len(df_modern):,}")
print(f"   Players: {df_modern['player_name'].nunique():,}")
print(f"   Years: {df_modern['year_id'].min()} to {df_modern['year_id'].max()}")

# 2. PER ERA DATASET (1978-2024) - PER and Win Shares available
print("\n2. Creating PER Era Dataset (1978-2024)...")
df_per_era = df_filtered[df_filtered['year_id'] >= 1978].copy()

# Remove records with missing PER or Win Shares
df_per_era = df_per_era.dropna(subset=['per', 'ws'])

print(f"   Records: {len(df_per_era):,}")
print(f"   Players: {df_per_era['player_name'].nunique():,}")
print(f"   Years: {df_per_era['year_id'].min()} to {df_per_era['year_id'].max()}")
```

```

# 3. COMPLETE DATASET - All eras with basic stats
print("\n3. Creating Complete Historical Dataset...")
df_complete = df_filtered.copy()

print(f"   Records: {len(df_complete):,}")
print(f"   Players: {df_complete['player_name'].nunique():,}")
print(f"   Years: {df_complete['year_id'].min()} to {df_complete['year_id'].
    ↪max():,}")

# Add dataset indicators
df_modern['dataset'] = 'modern'
df_per_era['dataset'] = 'per_era'
df_complete['dataset'] = 'complete'

print(f"\n" + "="*60)
print("DATASET COMPARISON")
print("="*60)

datasets = {
    'Modern Era (1993-2024)': df_modern,
    'PER Era (1978-2024)': df_per_era,
    'Complete Historical': df_complete
}

for name, data in datasets.items():
    print(f"\n{name}:")
    print(f"   Records: {len(data):,}")
    print(f"   Players: {data['player_name'].nunique():,}")
    print(f"   Years: {data['year_id'].min()}-{data['year_id'].max()}")
    print(f"   Avg seasons per player: {len(data) / data['player_name'].
    ↪nunique():.1f}")

    # Check stat availability
    stats_available = {}
    for stat in ['per', 'bpm', 'vorp', 'ws', 'ws_per_48']:
        available = data[stat].notna().sum()
        stats_available[stat] = f"{available:,{len(data):,} ({available/
    ↪len(data)*100:.1f}%)"

    print(f"   PER: {stats_available['per']}")
    print(f"   BPM: {stats_available['bpm']}")
    print(f"   WS: {stats_available['ws']}")

# Save datasets
print(f"\n" + "="*60)
print("SAVING DATASETS")

```

```

print("="*60)

# Save to CSV files
df_modern.to_csv('../data/03-cleaned/nba_modern_era_clean.csv', index=False)
print(" Modern era dataset saved to: ../data/03-cleaned/nba_modern_era_clean.
↳csv")

df_per_era.to_csv('../data/03-cleaned/nba_per_era_clean.csv', index=False)
print(" PER era dataset saved to: ../data/03-cleaned/nba_per_era_clean.csv")

df_complete.to_csv('../data/03-cleaned/nba_complete_clean.csv', index=False)
print(" Complete dataset saved to: ../data/03-cleaned/nba_complete_clean.csv")

# Save career statistics
career_stats_filtered.to_csv('../data/03-cleaned/nba_career_stats.csv')
print(" Career statistics saved to: ../data/03-cleaned/nba_career_stats.csv")

print(f"\nAll datasets saved successfully!")
print(f"Ready for peak analysis!")

# Display recommendations
print(f"\n" + "="*60)
print("ANALYSIS RECOMMENDATIONS")
print("="*60)
print("1. Use MODERN ERA dataset for comprehensive peak analysis")
print("    - Full advanced stats available")
print("    - Most reliable and consistent data")
print("    - Covers last 30+ years of NBA")
print("")
print("2. Use PER ERA dataset for longer historical trends")
print("    - PER and Win Shares available")
print("    - Covers 45+ years of NBA")
print("    - Good for era comparisons")
print("")
print("3. Use COMPLETE dataset for basic career pattern analysis")
print("    - Games, minutes, age data available")
print("    - Full NBA history")
print("    - Focus on playing time and longevity patterns")

```

===== CREATING ANALYSIS-READY DATASETS =====

1. Creating Modern Era Dataset (1993-2024)...
- Records: 10,596
- Players: 1,468
- Years: 1993 to 2025
2. Creating PER Era Dataset (1978-2024)...

Records: 14,146
Players: 1,821
Years: 1978 to 2025

3. Creating Complete Historical Dataset...

Records: 14,632
Players: 1,821
Years: 1900 to 2025

DATASET COMPARISON

Modern Era (1993-2024):

Records: 10,596
Players: 1,468
Years: 1993-2025
Avg seasons per player: 7.2
PER: 10,596/10,596 (100.0%)
BPM: 10,596/10,596 (100.0%)
WS: 10,596/10,596 (100.0%)

PER Era (1978-2024):

Records: 14,146
Players: 1,821
Years: 1978-2025
Avg seasons per player: 7.8
PER: 14,146/14,146 (100.0%)
BPM: 14,146/14,146 (100.0%)
WS: 14,146/14,146 (100.0%)

Complete Historical:

Records: 14,632
Players: 1,821
Years: 1900-2025
Avg seasons per player: 8.0
PER: 14,632/14,632 (100.0%)
BPM: 14,632/14,632 (100.0%)
WS: 14,632/14,632 (100.0%)

SAVING DATASETS

Modern era dataset saved to: ../data/03-cleaned/nba_modern_era_clean.csv
PER era dataset saved to: ../data/03-cleaned/nba_per_era_clean.csv
Complete dataset saved to: ../data/03-cleaned/nba_complete_clean.csv
Career statistics saved to: ../data/03-cleaned/nba_career_stats.csv

All datasets saved successfully!
Ready for peak analysis!

=====

ANALYSIS RECOMMENDATIONS

=====

1. Use MODERN ERA dataset for comprehensive peak analysis
 - Full advanced stats available
 - Most reliable and consistent data
 - Covers last 30+ years of NBA
2. Use PER ERA dataset for longer historical trends
 - PER and Win Shares available
 - Covers 45+ years of NBA
 - Good for era comparisons
3. Use COMPLETE dataset for basic career pattern analysis
 - Games, minutes, age data available
 - Full NBA history
 - Focus on playing time and longevity patterns

```
[ ]: # Data Visualization and Exploration
print("="*60)
print("VISUALIZING CLEANED DATASET")
print("="*60)

# Set up the plotting environment
plt.style.use('default')
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('NBA Dataset Overview After Cleaning', fontsize=16,
             fontweight='bold')

# 1. Records by era
era_counts = df_complete['era'].value_counts()
axes[0, 0].bar(era_counts.index, era_counts.values, color='skyblue')
axes[0, 0].set_title('Records by Era')
axes[0, 0].set_xlabel('Era')
axes[0, 0].set_ylabel('Number of Records')
axes[0, 0].tick_params(axis='x', rotation=45)

# 2. Records by year (modern era)
modern_by_year = df_modern.groupby('year_id').size()
axes[0, 1].plot(modern_by_year.index, modern_by_year.values, marker='o',
                 linewidth=2)
axes[0, 1].set_title('Records by Year (Modern Era)')
axes[0, 1].set_xlabel('Year')
axes[0, 1].set_ylabel('Number of Records')
```

```

axes[0, 1].grid(True, alpha=0.3)

# 3. Career length distribution
career_lengths = career_stats_filtered['seasons']
axes[0, 2].hist(career_lengths, bins=20, color='lightgreen', alpha=0.7,
    edgecolor='black')
axes[0, 2].set_title('Career Length Distribution')
axes[0, 2].set_xlabel('Seasons Played')
axes[0, 2].set_ylabel('Number of Players')
axes[0, 2].axvline(career_lengths.mean(), color='red', linestyle='--',
    label=f'Mean: {career_lengths.mean():.1f}')
axes[0, 2].legend()

# 4. Games per season distribution
axes[1, 0].hist(df_complete['games'], bins=30, color='orange', alpha=0.7,
    edgecolor='black')
axes[1, 0].set_title('Games per Season Distribution')
axes[1, 0].set_xlabel('Games')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].axvline(df_complete['games'].mean(), color='red', linestyle='--',
    label=f'Mean: {df_complete["games"].mean():.1f}')
axes[1, 0].legend()

# 5. Minutes per game distribution
axes[1, 1].hist(df_complete['mpg'], bins=30, color='purple', alpha=0.7,
    edgecolor='black')
axes[1, 1].set_title('Minutes per Game Distribution')
axes[1, 1].set_xlabel('Minutes per Game')
axes[1, 1].set_ylabel('Frequency')
axes[1, 1].axvline(df_complete['mpg'].mean(), color='red', linestyle='--',
    label=f'Mean: {df_complete["mpg"].mean():.1f}')
axes[1, 1].legend()

# 6. Age distribution
axes[1, 2].hist(df_complete['age'], bins=25, color='coral', alpha=0.7,
    edgecolor='black')
axes[1, 2].set_title('Age Distribution')
axes[1, 2].set_xlabel('Age')
axes[1, 2].set_ylabel('Frequency')
axes[1, 2].axvline(df_complete['age'].mean(), color='red', linestyle='--',
    label=f'Mean: {df_complete["age"].mean():.1f}')
axes[1, 2].legend()

plt.tight_layout()
plt.show()

# Advanced stats distribution (modern era only)

```

```

print("\n" + "="*60)
print("ADVANCED STATS DISTRIBUTIONS (MODERN ERA)")
print("="*60)

fig, axes = plt.subplots(2, 2, figsize=(15, 10))
fig.suptitle('Advanced Stats Distributions (Modern Era 1993-2024)',
             ↪fontsize=16, fontweight='bold')

# PER distribution
per_data = df_modern['per'].dropna()
axes[0, 0].hist(per_data, bins=30, color='lightblue', alpha=0.7,
                 ↪edgecolor='black')
axes[0, 0].set_title('Player Efficiency Rating (PER)')
axes[0, 0].set_xlabel('PER')
axes[0, 0].set_ylabel('Frequency')
axes[0, 0].axvline(per_data.mean(), color='red', linestyle='--',
                   label=f'Mean: {per_data.mean():.1f}')
axes[0, 0].axvline(15, color='green', linestyle='--',
                   label='League Average: 15.0')
axes[0, 0].legend()

# BPM distribution
bpm_data = df_modern['bpm'].dropna()
axes[0, 1].hist(bpm_data, bins=30, color='lightcoral', alpha=0.7,
                 ↪edgecolor='black')
axes[0, 1].set_title('Box Plus/Minus (BPM)')
axes[0, 1].set_xlabel('BPM')
axes[0, 1].set_ylabel('Frequency')
axes[0, 1].axvline(bpm_data.mean(), color='red', linestyle='--',
                   label=f'Mean: {bpm_data.mean():.1f}')
axes[0, 1].axvline(0, color='green', linestyle='--',
                   label='League Average: 0.0')
axes[0, 1].legend()

# Win Shares distribution
ws_data = df_modern['ws'].dropna()
axes[1, 0].hist(ws_data, bins=30, color='lightgreen', alpha=0.7,
                 ↪edgecolor='black')
axes[1, 0].set_title('Win Shares (WS)')
axes[1, 0].set_xlabel('Win Shares')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].axvline(ws_data.mean(), color='red', linestyle='--',
                   label=f'Mean: {ws_data.mean():.1f}')
axes[1, 0].legend()

# Win Shares per 48 distribution
ws48_data = df_modern['ws_per_48'].dropna()

```

```

axes[1, 1].hist(ws48_data, bins=30, color='gold', alpha=0.7, edgecolor='black')
axes[1, 1].set_title('Win Shares per 48 (WS/48)')
axes[1, 1].set_xlabel('WS/48')
axes[1, 1].set_ylabel('Frequency')
axes[1, 1].axvline(ws48_data.mean(), color='red', linestyle='--',
                    label=f'Mean: {ws48_data.mean():.3f}')
axes[1, 1].axvline(0.100, color='green', linestyle='--',
                    label='Good: 0.100')
axes[1, 1].legend()

plt.tight_layout()
plt.show()

# Summary statistics
print("\nSummary Statistics (Modern Era):")
print("="*40)
stats_summary = df_modern[['per', 'bpm', 'ws', 'ws_per_48']].describe()
print(stats_summary.round(3))

```

```

[21]: # Final Summary and Next Steps
print("="*80)
print("DATA CLEANING AND FILTERING COMPLETE")
print("="*80)

print("SUMMARY OF WORK COMPLETED:")
print("="*40)
print(" Loaded complete NBA dataset (35,647 raw records)")
print(" Removed invalid records (DNP, zero games/minutes)")
print(" Handled duplicate player-season records")
print(" Applied filtering criteria for meaningful analysis")
print(" Created era-based dataset splits")
print(" Generated analysis-ready datasets")
print(" Performed data quality validation")
print(" Created visualizations for data exploration")

print(f"\nFINAL DATASET SUMMARY:")
print("="*40)
print(f" MODERN ERA (1993-2024): {len(df_modern):,} records, {len(df_modern['player_name'].unique()):,} players")
print(f" PER ERA (1978-2024): {len(df_per_era):,} records, {len(df_per_era['player_name'].unique()):,} players")
print(f" COMPLETE (1946-2024): {len(df_complete):,} records, {len(df_complete['player_name'].unique()):,} players")

print(f"\nDATA QUALITY METRICS:")
print("="*40)

```

```

print(f" Average career length: {career_stats_filtered['seasons'].mean():.1f}␣
↳seasons")
print(f" Average games per season: {df_complete['games'].mean():.1f}")
print(f" Average minutes per game: {df_complete['mpg'].mean():.1f}")
print(f" Age range: {df_complete['age'].min()}-{df_complete['age'].max()}␣
↳years")

print(f"\nFILES CREATED:")
print("="*40)
print(" ../data/03-cleaned/nba_modern_era_clean.csv")
print(" ../data/03-cleaned/nba_per_era_clean.csv")
print(" ../data/03-cleaned/nba_complete_clean.csv")
print(" ../data/03-cleaned/nba_career_stats.csv")

print(f"\nNEXT STEPS - PEAK ANALYSIS:")
print("="*40)
print("1. Analyze peak patterns by age for each advanced stat")
print("2. Identify position-specific peak patterns")
print("3. Classify players by talent level (All-Star, role players, etc.)")
print("4. Create peak age visualizations and career arc charts")
print("5. Identify factors that influence peak timing")
print("6. Generate insights for front office decision making")

print(f"\nRECOMMENDED ANALYSIS WORKFLOW:")
print("="*40)
print("→ Start with MODERN ERA dataset for initial peak analysis")
print("→ Use PER ERA dataset for historical trend validation")
print("→ Focus on PER, BPM, and Win Shares as primary metrics")
print("→ Create age-based performance curves for each metric")
print("→ Segment analysis by position and playing time")

print(f"\n DATASET IS NOW READY FOR PEAK ANALYSIS!")
print("="*80)

```

```

=====
DATA CLEANING AND FILTERING COMPLETE
=====

```

```

SUMMARY OF WORK COMPLETED:
=====

```

```

Loaded complete NBA dataset (35,647 raw records)
Removed invalid records (DNP, zero games/minutes)
Handled duplicate player-season records
Applied filtering criteria for meaningful analysis
Created era-based dataset splits
Generated analysis-ready datasets
Performed data quality validation
Created visualizations for data exploration

```

FINAL DATASET SUMMARY:

=====

MODERN ERA (1993-2024): 10,596 records, 1,468 players
PER ERA (1978-2024): 14,146 records, 1,821 players
COMPLETE (1946-2024): 14,632 records, 1,821 players

DATA QUALITY METRICS:

=====

Average career length: 8.0 seasons
Average games per season: 65.4
Average minutes per game: 24.7
Age range: 18-43 years

FILES CREATED:

=====

../data/03-cleaned/nba_modern_era_clean.csv
../data/03-cleaned/nba_per_era_clean.csv
../data/03-cleaned/nba_complete_clean.csv
../data/03-cleaned/nba_career_stats.csv

NEXT STEPS - PEAK ANALYSIS:

=====

1. Analyze peak patterns by age for each advanced stat
2. Identify position-specific peak patterns
3. Classify players by talent level (All-Star, role players, etc.)
4. Create peak age visualizations and career arc charts
5. Identify factors that influence peak timing
6. Generate insights for front office decision making

RECOMMENDED ANALYSIS WORKFLOW:

=====

- Start with MODERN ERA dataset for initial peak analysis
- Use PER ERA dataset for historical trend validation
- Focus on PER, BPM, and Win Shares as primary metrics
- Create age-based performance curves for each metric
- Segment analysis by position and playing time

DATASET IS NOW READY FOR PEAK ANALYSIS!

=====